

RAG

Содержание

- 1. Обзор RAG
- 2. Смена парадигм RAG
- 3. Ключевые технологии и оценивание
- 4. Стек RAG и практика в индустрии
- 5. Итоги и перспективы

Предпосылки

Недостатки LLM

- Галлюцинации
- Устаревшая информация
- Низкая эффективность параметризации знаний
- Недостаток глубинных знаний в специализированных доменах
- Слабые выводные способности (inference)

Практические требования к применению

- Точные ответы для конкретной предметной области
- Частые обновления данных
- Трассируемость и объяснимость сгенерированного контента
- Управляемая стоимость
- Защита конфиденциальности данных

Сравнение: RAG vs Fine-Tuning

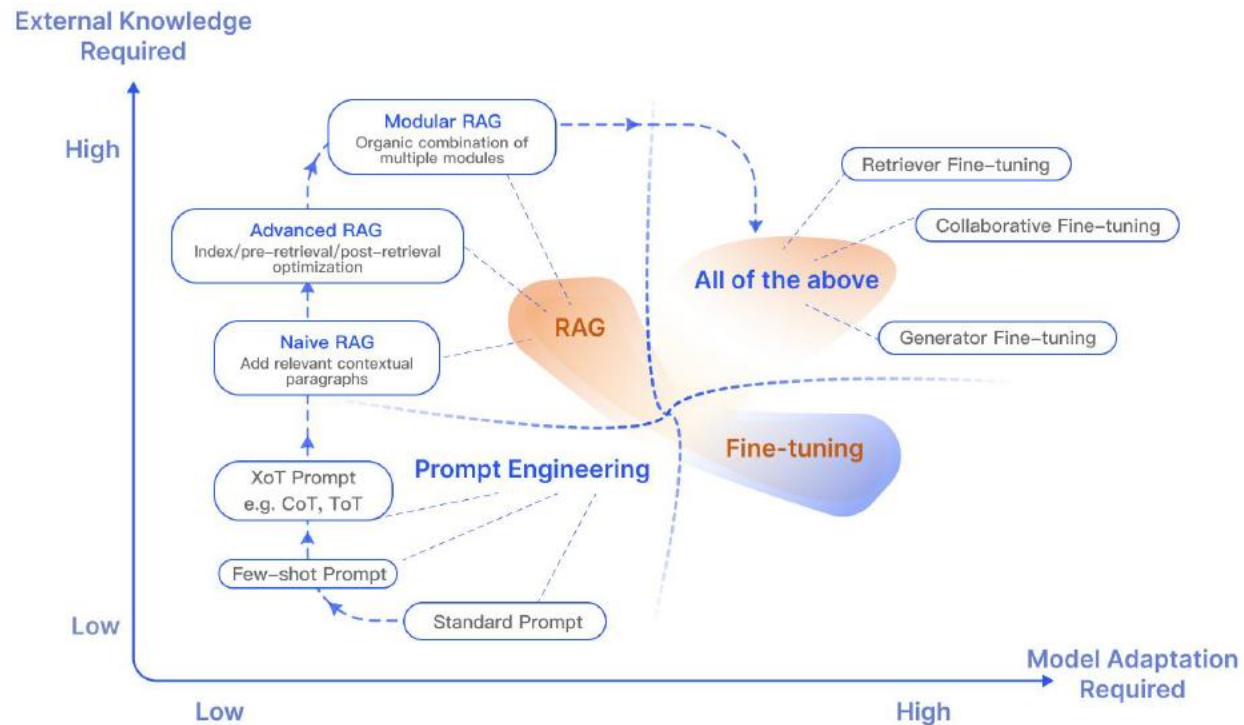
Сравнение характеристик	RAG	Fine-Tuning (дообучение)
Обновление знаний	Прямое обновление базы извлекаемых знаний обеспечивает актуальность информации без необходимости частых дообучений, что делает подход удобным для сред с динамичными данными.	Хранит статические данные; для обновления знаний и данных требуется повторное дообучение.
Внешние знания	Хорошо использует внешние ресурсы, особенно подходит для доступа к документам и другим структурированным/неструктурированным базам данных.	Можно использовать для согласования внешних знаний, полученных на этапе предобучения больших языковых моделей, но менее практично для часто меняющихся источников данных.
Обработка данных	Требует минимальной обработки и подготовки данных.	Зависит от создания высококачественных датасетов; ограниченные наборы данных могут не дать значимого улучшения качества.
Настройка модели	Фокусируется на поиске информации и интеграции внешних знаний, но не всегда позволяет полноценно настроить поведение модели или стиль изложения.	Позволяет настраивать поведение LLM, стиль письма и отраслевые знания под конкретные тоны и термины.
Интерпретируемость	Ответы можно отследить до конкретных источников данных, что повышает интерпретируемость и трассируемость.	Похоже на «чёрный ящик»: не всегда ясно, почему модель реагирует тем или иным образом; интерпретируемость относительно ниже.
Вычислительные ресурсы	Зависит от ресурсов для поддержки стратегий извлечения и технологий, связанных с базами данных. Дополнительно требует сопровождения интеграций с внешними источниками и их обновлений.	Необходимы подготовка и курирование высококачественных обучающих наборов, определение целей дообучения и соответствующие вычислительные ресурсы.

Пути оптимизации LLMs

Prompt Engineering

Retrieval-Augmented Generation

Instruct / Fine-tuning



A typical case of RAG

Недостатки LLM

- Галлюцинации
- Устаревшая информация
- Низкая эффективность параметризации знаний
- Недостаток глубинных знаний в специализированных доменах
- Слабые выводные способности

Практические требования к приложениям

- Точные ответы в предметной области
- Частые обновления данных
- Трассируемость и объяснимость сгенерированного контента
- Контролируемая стоимость
- Защита конфиденциальности данных

Типичный пример RAG

- При ответе на вопросы или генерации текста система сначала извлекает релевантную информацию из большого числа документов, затем LLM генерирует ответы на основе этого контекста.
- Подключение внешней базы знаний устраняет необходимость переобучать большую модель под каждую задачу.
- RAG особенно полезна для задач, требующих большого объёма знаний.

Способы улучшения LLM

- Инжиниринг подсказок (Prompt Engineering)
- Генерация с расширением за счёт поиска (RAG)
- Инструкционное обучение / Дообучение (Fine-tuning)
- Символические знания или параметрические знания

RAG против дообучения

- RAG: подключение внешних знаний без изменения параметров модели.
- Fine-tuning: обновление параметров модели под конкретные задачи и данные.
- Комбинированные подходы улучшают обобщающую способность и контроль затрат.

Применения RAG — ключевые сценарии

- «Длинный хвост» данных
- Частые обновления знаний
- Ответы с проверкой и трассируемостью
- Специализированные предметные области
- Конфиденциальность данных

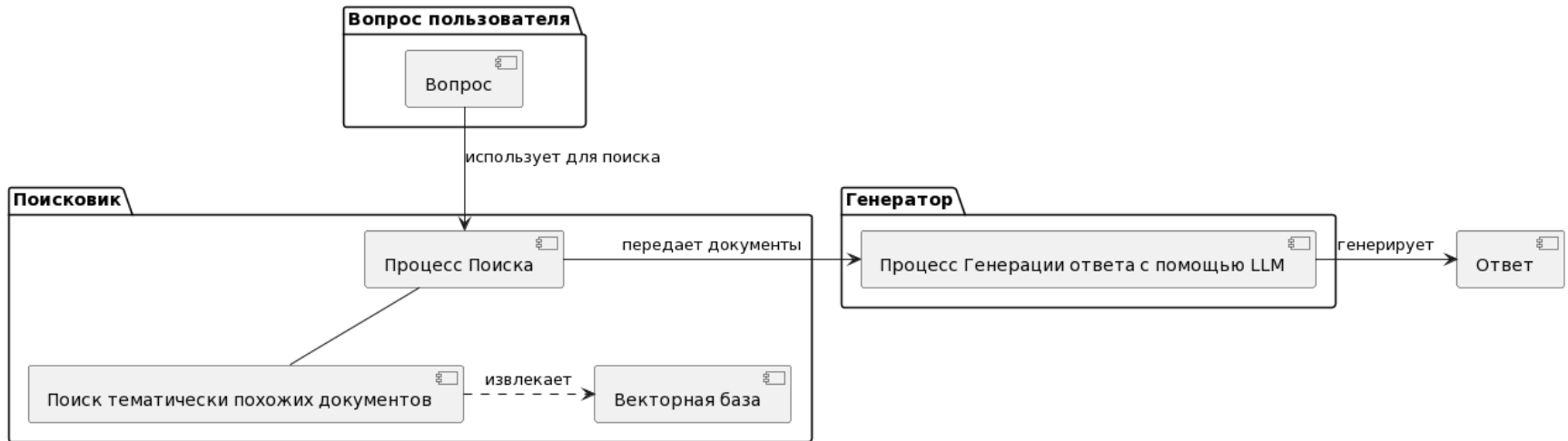
Примеры задач

- Вопрос–ответ (Q&A); Фактчекинг; Резюмирование; Диалог; Логический вывод на естественном языке (NLI).
- Генерация кода; Машинный перевод; Анализ тональности; Здравый смысл (commonsense) рассуждения.
- Интернет-расширённая генерация; Документное доказательство; Документ-ориентированный prompting.

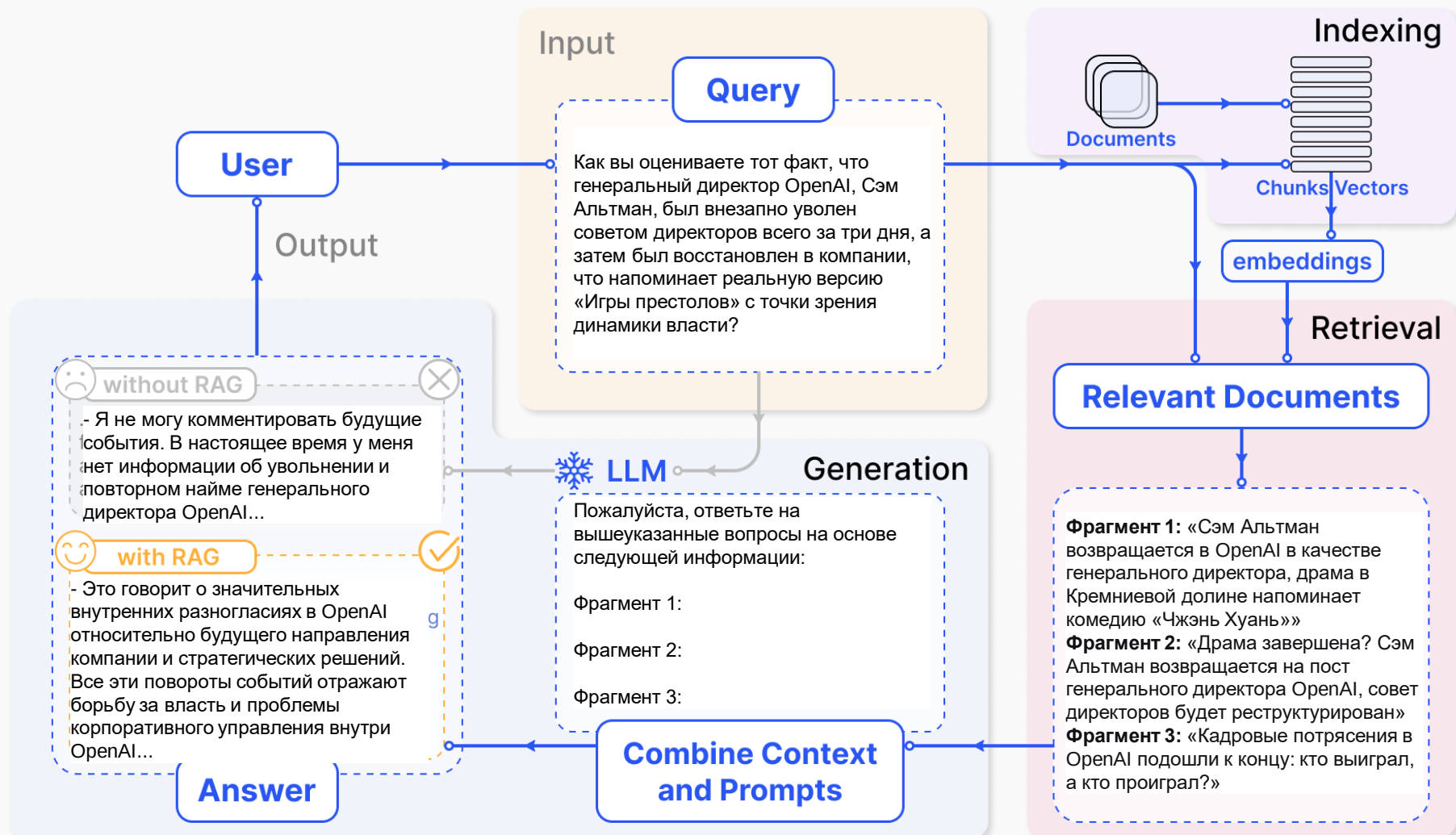
Часть 02

Смена парадигм RAG

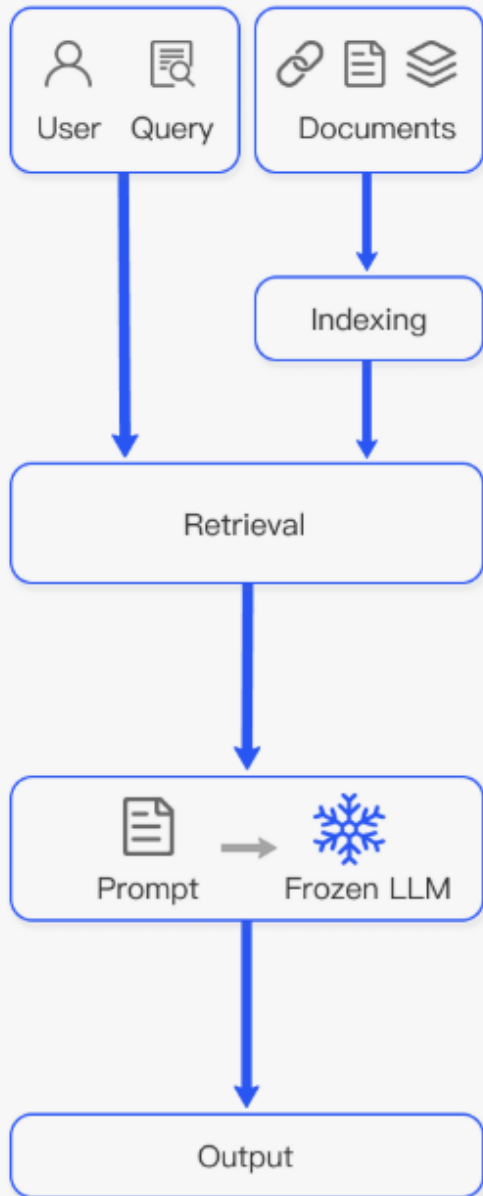
Архитектура RAG



Архитектура RAG



Базовая (Naive) RAG — шаги



Naive RAG

- Шаг 1. Индексация:
 - • Разделение документов на фрагменты (chunks).
 - • Получение эмбеддингов для каждого фрагмента.
 - • Сохранение в векторную БД.
- Шаг 2. Поиск (Retrieval):
 - • Извлечение k наиболее релевантных фрагментов по косинусной близости и др.
- Шаг 3. Генерация:
 - • Объединение запроса и найденного контекста; ввод в LLM; получение ответа.

► Наивный RAG

Шаг 1 Индексация

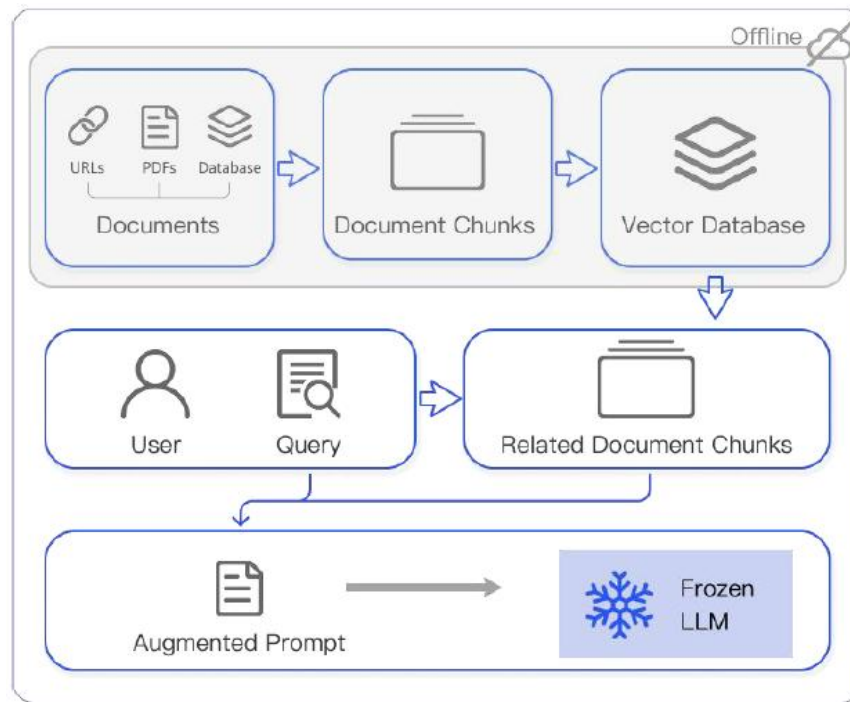
1. Разделите документ на равные фрагменты (chunks), каждый фрагмент — это часть исходного текста.
2. Используйте модель кодировок для генерации эмбединга для каждого фрагмента.
3. Сохраните эмбединг каждого блока во векторной базе данных.

Шаг 2 Извлечение

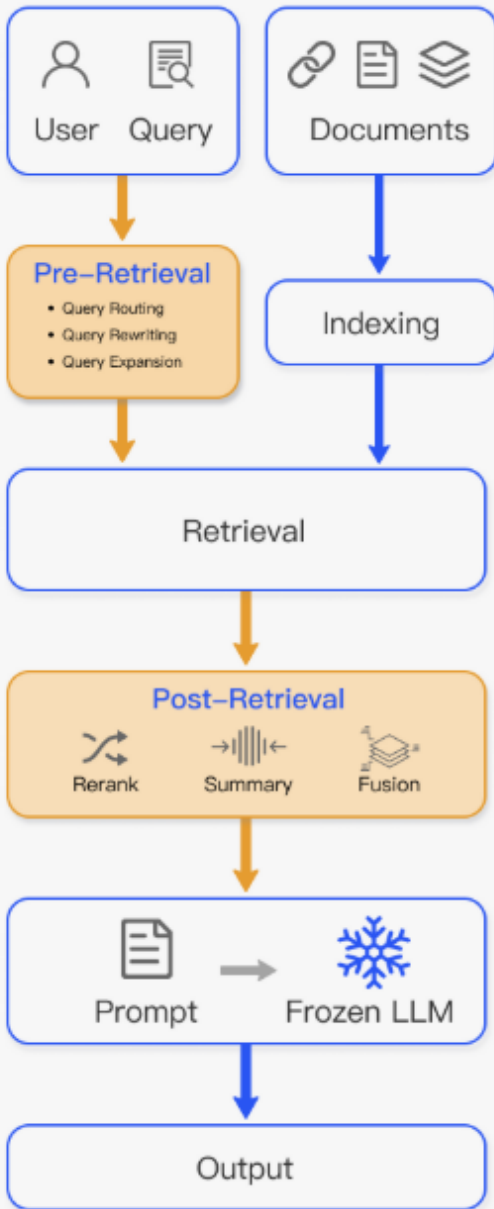
Извлеките k наиболее релевантных документов с помощью поиска по сходству векторных представлений.

Шаг 3 Генерация

Исходный запрос и извлечённый текст объединяются и подаются в LLM, чтобы получить итоговый ответ.



Легенда схемы: Documents → Документы; Document Chunks → Фрагменты документа; Vector Database → Векторная база данных; User → Пользователь; Query → Запрос; Related Document Chunks → Соответствующие фрагменты документов; Augmented Prompt → Расширенный промпт; Frozen LLM → Замороженная LLM.



Advanced RAG

Продвинутая RAG

Оптимизация индексации:

- Скользящее окно; тонкая сегментация; метаданные.

Предварительный этап (pre-retrieval):

- Маршруты поиска; суммаризация; переформулирование; оценка уверенности.

Постобработка (post-retrieval):

- Переранжирование; фильтрация; нормализация контента.

Конвейер: Оптимизация индекса →
Предпоиск → Поиск → Постобработка
→ Генерация

Продвинутый RAG

Оптимизация индекса → Процесс до извлечения → Извлечение →
Процесс после извлечения → Генерация

- **Оптимизация индексации данных:**

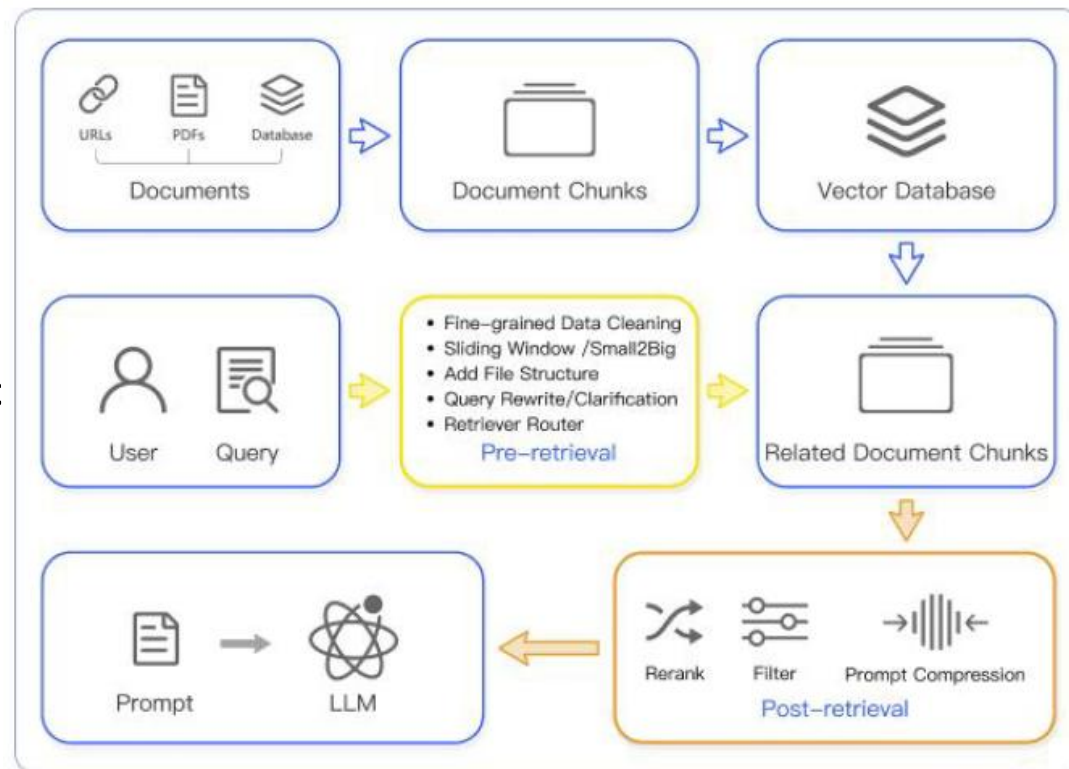
скользящее окно, тонкая (fine-grained) сегментация, добавление метаданных

- **Процесс до извлечения:**

маршрутизация извлечения (routes), суммаризации, переформулирование запросов, оценка уверенности

- **Процесс после извлечения:**

переупорядочивание (rerank), фильтрация, сжатие промпта



Modules

The diagram illustrates the components of a Retrieval-Augmented Generation (RAG) system. The central module is **RAG**. Surrounding it are eight modules connected by dashed lines in a circular flow: **Search**, **Predict**, **Fusion**, **Memory**, **Demonstrate**, **Rewrite**, **Read**, and **Rerank**. The **Retrieve** module is positioned at the top, connected to **Search** and **Rerank**.

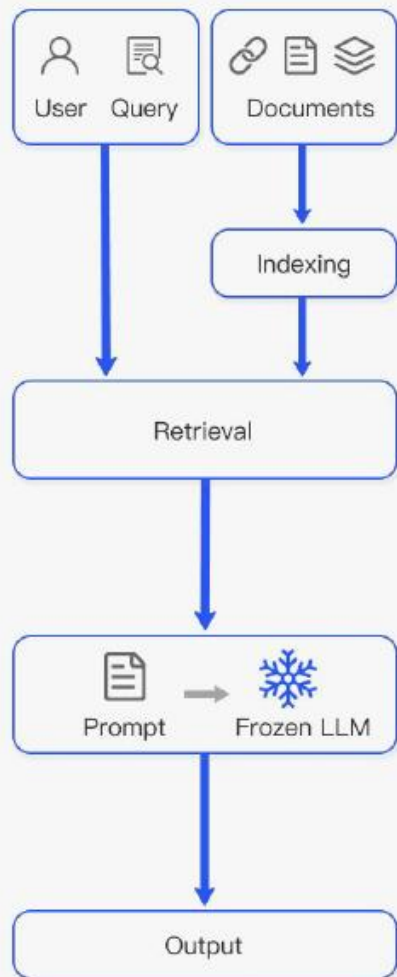
Patterns

The diagram shows four different RAG patterns:

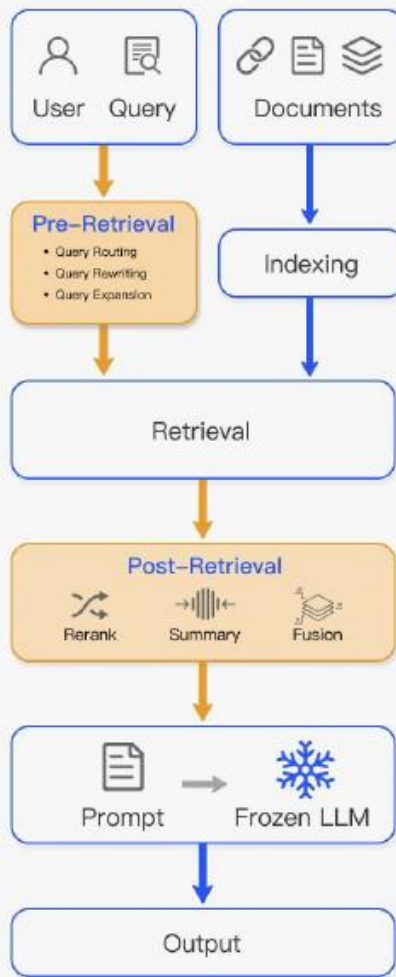
- Naive RAG**: A simple flow from **Retrieve** to **Read**.
- Advanced RAG**: A flow from **Rewrite** to **Retrieve**, then **Rerank**, and finally **Read**.
- DSP** (Khattab et al., 2022): A flow from **Demonstrate** to **Search**, then **Predict**.
- ITER-RETGEN** (Shao et al., 2023): A flow from **Retrieve** to **Read**, then **Retrieve** again, and finally **Read**. A feedback loop connects the final **Read** back to the first **Retrieve**.

- Модули: Чтение · Поиск · Переформулирование · Переранжирование · Фильтрация · Агрегация · Генерация · Рефлексия.
- Шаблоны: Retrieve-then-Read; Rewrite-Retrieve-Read; DSP (Demonstrate-Search-Predict).

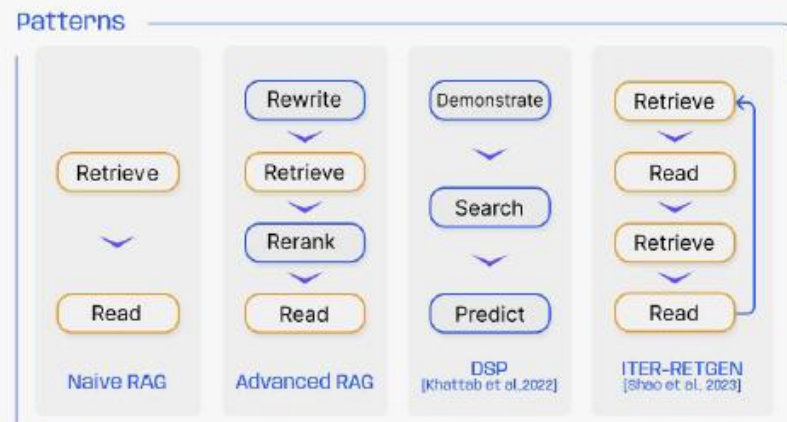
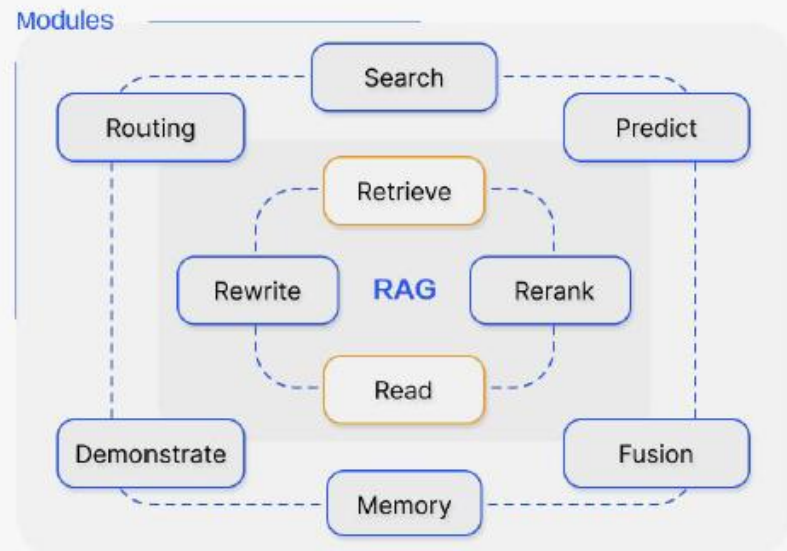
Сравнение RAG



Naive RAG



Advanced RAG



Modular RAG

Оптимизации поиска

1. Первоначальную обработку и очистку вопроса пользователя
2. Поиск данных в хранилищах
3. Ранжирование полученных результатов из хранилища
4. Обработка и комбинирование результатов в ответ
5. Оценка ответа
6. Применение форматирования, стилистики и тона

Первоначальную обработку и очистку вопроса

Цель: превратить «грязный» человеческий запрос в аккуратный, однозначный и пригодный для поиска.

- **Нормализуем текст:** убрать лишние пробелы, эмодзи, неинформативные служебные слова.
- **Определяем язык и тему:** чтобы потом взять правильные эмбединги/правильный индекс.
- **Выделяем сущности (NER):** даты, компании, продукты, города — это может пойти в фильтр поиска.
- **Расширяем/уточняем запрос (reformulation):** иногда запрос слишком короткий — «инструкция по командировкам». Мы превращаем его в: «Покажи действующую инструкцию по служебным командировкам для сотрудников в РФ, актуальную на сегодня».
- **Безопасность и политика:** вырезать то, что нельзя показывать (PII, мат и т.п.) — особенно если запрос пойдёт в внешний API.

Первоначальную обработку и очистку вопроса



You

Столица Лондона



ChatGPT

Столицей Великобритании является Лондон.

< 2 / 2 >



You

и?

Первоначальную обработку и очистку вопроса



You

Вот вопрос юзера. Думай по шагам и оцени что юзер хочет узнать. Отвечай в формате:

Почему я думаю, что юзер спросил именно это: <объяснение>

Поэтому реальный вопрос юзера: <вопрос>

Юзер:

Столица!!!! Омерики!!!!111



ChatGPT

Почему я думаю, что юзер спросил именно это: Пользователь использует множество восклицательных знаков и многоточий, что может указывать на его эмоциональное состояние и неопределенность относительно того, какую информацию он именно ищет. Вероятно, он хочет узнать столицу Соединенных Штатов Америки, но его запрос сформулирован несколько нестандартно.

Поэтому реальный вопрос юзера: Какая столица Америки?

Первоначальную обработку и очистку вопроса

Что еще может потребоваться:

- Создание похожих запросов
- Разбивка запроса на простые запросы
- Классификация запроса

Поиск данных в хранилищах

Цель: по отчищенному запросу найти векторно-подходящие куски текста.

- **Получаем эмбеддинг запроса** той же моделью, что и документы.
- **Делаем векторный поиск** по FAISS/Chroma/PGVector/Elastic+vector.
- **Применяем фильтры** по метаданным: дата, департамент, язык, уровень допуска.
- **(Опционально) обращаемся в несколько хранилищ** — документы, база Q&A, база продуктов, база тикетов.

Поиск данных в хранилищах

- **Dense Retriever** - использует обычно трансформеры, например BERT, для кодирования как запросов, так и документов в векторы в многомерном пространстве.
- **Sparse Retriever** - использует традиционные методы информационного поиска, такие как TF-IDF
- И много других
- Сходство между запросом и документом измеряется близостью их векторов в этом пространстве

Ранжирование полученных результатов из хранилища

Цель: из всех «примерно подходящих» кусочков выбрать те, по которым модель точно сможет ответить.

- **Сходство по смыслу (similarity score)** — косинус, dot-product.
- **Свежесть** — более новые документы выше.
- **Источник/надежность** — официальные регламенты выше, чем переписка в чате.
- **MMR (Maximal Marginal Relevance)** — чтобы не получить 5 почти одинаковых абзацев.
- **Ролевые/доступные документы** — показываем только то, что пользователь может видеть.
- **Cross Encoder** для ре-ранжирования полученных результатов и отбрасывания наименее релевантных (например достаем топ 30 результатов из векторной базы (top k), ранжируем Cross Encoder'ом, берем первые 10)
-

Обработка и комбинирование результатов в ответ

Цель: из k фрагментов сделать **один** связный контекст для LLM.

1. **Приводим фрагменты к единому виду:** текст, язык, кодировка.
2. **Урезаем/сжимаем** слишком длинные куски (иначе промпт раздуется).
3. **Группируем по источнику:** чтобы в ответе можно было сослаться «из Положения...».
4. **Строим промпт по шаблону:**
 - роль: «ты ассистент по внутренним документам компании»
 - контекст: [DOC 1] ... [DOC 2] ...
 - инструкция: «отвечай только по ним, если ответа нет — скажи, что нет»
 - вопрос: {user_question}

Оценка ответа

- **Цель:** понять, можно ли этот ответ показывать, или лучше переспросить/сжать/дослать источники.
- Что проверяем:
- **Groundedness / Faithfulness** — все ли факты взяты из контекста. Если модель пишет то, чего в контексте нет → флаг «сомнительно».
- **Полнота** — модель ответила на всё или только на 1 подпункт?
- **Стиль и политика** — нет ли запрещённого, утечки, персональных данных.
- **Уверенность** — можно посчитать разными способами:
 - модель отдельно генерирует «уверенность»;
 - смотрим на разброс similarity у извлечённых кусков;
 - считаем длину ответа vs длину контекста.
- **Fallback** — если уверенность низкая, делаем один из сценариев:
 - «Не нашёл точного ответа, вот документы, где это может быть»
 - «Уточните: для какого региона/должности/уровня?»
 - «Вот 2 варианта, выберите нужный»

Применение форматирования, стилистики и тона

Цель: привести итог к виду «как будто это написал сотрудник / сервис / бот компании».

- Что делаем:
- **Стиль:** «деловой», «для сотрудника», «для кандидата», «для руководителя».
- **Формат:** список, план, таблица, короткий ответ, шаблон письма.
- **Ссылки на источники:** «См. “Положение об адаптации, 2024”, п. 3.2».
- **Локализация:** язык, дата/валюта/отчество.
- **Контекст канала:** в чат — короче, в письмо — полнее, в Confluence — с заголовками.

Практика